

Testes de Software

Implementando Padrões de Teste (Test Patterns)

Júlia de Souza Ventura

Matrícula: 869002

1. Padrões de Criação de Dados (Builders)



Figura 1 - Estrutura de arquivos criados para a suíte de testes.

1.1 Por que CarrinhoBuilder e não CarrinhoMother?

Conforme apresentado nos slides da disciplina, o padrão Object Mother é uma classe que centraliza a criação de objetos em estados pré-configurados, "uma mãe que te entrega seus objetos prontos para brincar". Esse padrão funciona muito bem para entidades simples e estáveis. No projeto, o User tem poucos atributos e os cenários de teste se resumem a dois tipos fixos: padrão e premium. Por isso, o UserMother com dois métodos estáticos resolve o problema perfeitamente.

O Carrinho, porém, é uma entidade muito variável: pode ter usuários diferentes, nenhum item, vários itens, valores distintos. Aplicar o Object Mother aqui levaria a uma "explosão de variações", onde seria necessário criar um método diferente para cada combinação possível, como `umCarrinhoVazioParaUsuarioPremium()`, tornando a classe difícil de manter.

1.2 Antes e Depois do Data Builder

```
// Antes - setup manual (Obscure Setup)
const user = new User(2, 'Usuario Premium', 'premium@email.com', 'PREMIUM');
const item1 = new Item('Notebook', 100);
const item2 = new Item('Mouse', 100);
const carrinho = new Carrinho(user, [item1, item2]);
```

O setup acima tem mais código de construção do que de teste, exatamente o smell de Obscure Setup.

```
// Depois — com Data Builder
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([new Item('Notebook', 100), new Item('Mouse', 100)])
  .build();
```

Com o Builder, apenas o que é relevante para o cenário fica visível. Os valores padrão cuidam do resto, deixando o teste mais fácil de ler e de entender.

```
juliaventura@Julias-MacBook-Pro test-pattern % npm test

> test-pattern@1.0.0 test
> jest

PASS  __tests__/_CheckoutService.test.js
  CheckoutService
    quando o pagamento falha
      ✓ deve retornar null e não salvar o pedido nem enviar e-mail (2 ms)
    quando um cliente Premium finaliza a compra
      ✓ deve aplicar 10% de desconto, salvar o pedido e enviar e-mail de confirmação

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 0.385 s, estimated 1 s
Ran all test suites.
juliaventura@Julias-MacBook-Pro test-pattern %
```

Figura 2 - Execução da suíte de testes com todos os cenários passando.

1.3 Como o Builder melhora a legibilidade e manutenção

Em termos de **legibilidade**, os métodos encadeados funcionam como uma frase em linguagem natural. Quem lê o teste entende imediatamente a intenção do cenário sem precisar decodificar construtores com vários parâmetros. Em termos de **manutenção**, se a classe Carrinho mudar como por exemplo, ganhar um campo de cupom de desconto, a correção fica centralizada no CarrinhoBuilder. Não é preciso alterar todos os testes que criam carrinhos. O Builder age como uma camada de proteção entre os testes e a estrutura interna do domínio.

2. Padrões de Test Doubles (Mocks vs. Stubs)

2.1 O cenário: sucesso para cliente Premium

O teste escolhido para análise é o de sucesso para um cliente Premium, que verifica se o checkout aplica corretamente o desconto de 10%, salva o pedido e envia o e-mail de confirmação. Nesse teste, três dependências externas foram substituídas por dublês: *GatewayPagamento*, *PedidoRepository* e *EmailService*.

2.2 GatewayPagamento - Stub

```
const gatewayStub = {
  cobrar: jest.fn().mockResolvedValue({ success: true })
};
```

O GatewayPagamento foi usado como Stub porque o objetivo não é verificar se ele foi chamado, mas sim controlar o fluxo do sistema: ao retornar { success: true }, garantimos que o código avança para as próximas etapas. O que o teste verifica é o estado resultante, se o valor cobrado foi R\$180,00, aplicando corretamente os 10% de desconto sobre R\$200,00. Isso é a Verificação de Estado (State Verification).

```
expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, cartaoCredito);
```

2.3 EmailService - Mock

```
const emailMock = {  
  enviarEmail: jest.fn().mockResolvedValue(undefined)  
};
```

O EmailService foi usado como Mock porque o envio de e-mail é um efeito colateral do processo de checkout onde não há valor de retorno relevante para o teste. O que importa é verificar se a interação ocorreu corretamente: o e-mail foi enviado exatamente 1 vez, para o destinatário certo, com o assunto certo e com o ID do pedido na mensagem. Isso é a Verificação de Comportamento (Behavior Verification).

```
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);  
expect(emailMock.enviarEmail).toHaveBeenCalledWith(  
  'premium@email.com',  
  'Seu Pedido foi Aprovado!',  
  expect.stringContaining('42')  
);
```

3. Conclusão

O uso deliberado de Padrões de Teste atua diretamente na prevenção de Test Smells. O CarrinhoBuilder elimina o smell de Obscure Setup ao tornar explícito apenas o que é relevante para cada cenário, centralizando a lógica de construção de objetos em um único lugar. Já o UserMother complementa isso para entidades simples, fornecendo instâncias nomeadas e semanticamente claras.

Os Test Doubles: Stubs e Mocks, resolvem o smell de Fragile Tests causado pelo acoplamento a dependências externas reais. Ao substituir o gateway de pagamento, o repositório e o serviço de e-mail por dublês controlados, os testes se tornam rápidos, determinísticos e isolados: rodam sem conexão com serviços externos e falham apenas quando a lógica de negócio está errada. Como apresentado nos slides da disciplina, padrões de teste não são apenas uma técnica, são uma mudança de mentalidade. Uma suíte sustentável é aquela que qualquer desenvolvedor consegue ler, entender e modificar com confiança. Os padrões estudados neste trabalho são o principal instrumento para isso.

[REFERÊNCIAS]

TAVARES, Cleiton. *Padrões de Teste (Test Patterns)*. Slides da disciplina Teste de Software — PUC Minas, 2025.